

GlazeCorp Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

Alix40

Arnie

October 31, 2026

Contents

1 About Cyfrin 2

2 About This Document 2

3 Disclaimer 2

4 Copyright and License 2

5 Audit Overview 3

6 Audit Scope 3

7 Risk Classification 4

8 Protocol Summary 4

9 Attestation 4

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for GlazeCorp on GlazeCorp. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report was delivered privately to GlazeCorp, who retain it as the full record of the engagement.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from GlazeCorp or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 10 days, the Cyfrin team conducted an audit on the GlazeCorp code provided by GlazeCorp. The findings consist of 3 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	GlazeCorp
Repository	glazecorp
Commit	8adbf5e6182d...
Fix Commit	bc3d5d7205c6...
Audit Timeline	Oct 12th - Oct 23rd, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	3	0
Informational	7	7	0
Gas Optimizations	1	1	0
Total	11	11	0

6 Audit Scope

The audit scope was limited to the following files:

```
packages/contracts/src/core/Auction.sol
packages/contracts/src/core/AuctionFactory.sol
packages/contracts/src/core/Bribe.sol
packages/contracts/src/core/BribeFactory.sol
packages/contracts/src/core/BribeRouter.sol
packages/contracts/src/core/Coin.sol
packages/contracts/src/core/Fund.sol
packages/contracts/src/core/FundController.sol
packages/contracts/src/core/Mine.sol
packages/contracts/src/core/Resonance.sol
packages/contracts/src/core/ResonanceRouter.sol
packages/contracts/src/core/SignalCoin.sol
packages/contracts/src/fullrange/FullRange.sol
packages/contracts/src/fullrange/FullRangeERC20.sol
packages/contracts/src/fullrange/FullRangeERC20Deployer.sol
packages/contracts/src/fullrange/FullRangeMath.sol
packages/contracts/src/launch/CoinDeployer.sol
packages/contracts/src/launch/ControllerDeployer.sol
packages/contracts/src/launch/FundDeployer.sol
packages/contracts/src/launch/Launchpad.sol
```

```
packages/contracts/src/launch/MineDeployer.sol
packages/contracts/src/launch/ResonanceDeployer.sol
packages/contracts/src/launch/RouterDeployer.sol
packages/contracts/src/launch/SignalDeployer.sol
packages/contracts/src/periphery/OperationsPeriphery.sol
packages/contracts/src/periphery/Periphery.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

GlazeCorp is a token launchpad where each launched Coin gets its own self-funding on-chain treasury. In one transaction, `Launchpad::launch` deploys a Coin, an ownerless `Fund`, a `Mine`, a `Resonance` signal market with its `ResonanceRouter`, a `SignalCoin` and a creator-owned `FundController`. It then seeds a Coin/base full-range Uniswap V4 position through the `FullRange` hook and permanently locks the initial LP shares.

`Mine` runs a single-slot Dutch auction paid in USDG. The winning miner earns Coin at a rate fixed for its tenure until it is replaced. The outgoing miner gets a claim for 80% of the replacement payment, optional creator and protocol fees are credited as claims, and the remainder is sent to the revenue destination.

`Resonance` streams that revenue to strategies according to the Coin signaled through `SignalCoin`. Each strategy is an `Auction` that sells its accumulated USDG for a payment token. The payment goes to the `Fund`, except for an optional share sent to the strategy's `BribeRouter`, which forwards it to a `Bribe` that rewards the strategy's signalers. Coin holders can redeem a selected in-kind basket of `Fund` assets pro rata to their share of `Mine::effectiveTotalSupply`.

`FullRange` is a shared V4 hook that manages one full-range position per ERC-20 wrapper (`FullRangeERC20`), with one raw share per unit of native liquidity. Wrappers for the same currency pair and fee share a PoolId but keep independent positions and fee checkpoints. Realized swap fees are held as ERC-6909 claims. They are split 70/30 between the wrapper's creator recipients and the protocol recipient, or go entirely to the creator recipients while no protocol recipient is set.

All contracts are non-upgradeable. Each launch's Coin, `Mine`, `Resonance` and LP wrapper are administered by the creator through `FundController`. The `Launchpad` and `FullRange` owners only control global protocol fee settings.

9 Attestation

Cyfrin attests that it performed a security review of the GlazeCorp code provided by GlazeCorp.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-10-31-cyfrin-glazecorp-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to GlazeCorp has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=fab14783d1a520e6596385abdef12bdc47d43161334be9a92253098565b74d38
F=2026-10-31-cyfrin-glazecorp-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.